

Implementations using Special purpose DSP processors:

Digital Signal Processor or DSP processor are used to implement and execute DSP algorithms in real time (often real-time implies 'as soon as possible', but within specified time limits).

a) General purpose DSP processors:

these are basically high speed microprocessors with hardware and instruction sets optimized for DSP operations.

Examples of such processors include fixed-point devices such as Texas instruments TMS320C54x and Motorola DSP563x processors, and floating point processors such as Texas instruments TMS320C6x and analog devices ADSP21xxx SHARC processors.

b) Special-purpose DSP processors:

these include (i) hardware designed for efficient execution of specific DSP algorithms (sometimes called algorithm-specific hardware), e.g. FFT and

(ii) hardware designed for specific applications (sometimes called application specific processors), e.g. for PCM in telecommunication or audio applications.

Example of Special purpose DSP processors are Cirrus's processor for digital audio sampling rate converters (CS8420), Mitel's multi-channel telephony voice echo canceller (MT9300), FFT processor (PDSP16515A) and programmable FIR filter (VPDSP1625).

~~on overall architecture~~

~~Imp~~

Pipelined Implementations:

This is a technique used extensively in DSP to increase speed as it allows two or more operations to overlap during execution. In pipelining, a task is broken down into a no. of distinct sub-tasks which are then overlapped during execution.

A pipeline is ~~a kind of~~ ^{akin to} a typical production line in a factory, such as a car or TV assembly plant. As in the production line, the task is broken down into small, independent sub-tasks called pipe stages which are connected in series to form a pipe. Execution is sequential.

Instruction 1	P_1	P_2	P_3	$P_1 = \text{Pipe Stage 1}$
Instruction 2		P_1	P_2	$P_2 = \text{Pipe Stage 2}$
Instruction 3			P_1	$P_3, P_3 = \text{Pipe Stage 3}$

Fig. (a)

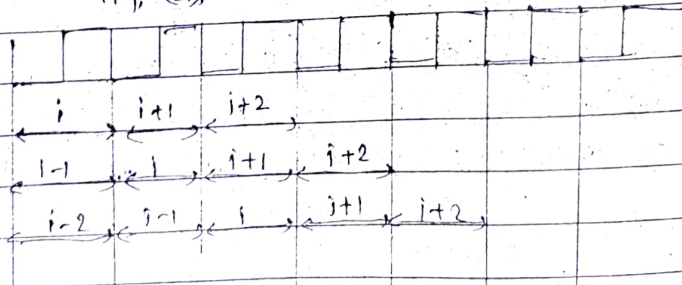


Fig. (b)

Fig. (a) Illustration of the Concepts of Pipelining

The fig: gives a timing diagram of a stage pipeline. Typically, each step in the pipeline takes one machine cycle to complete. Thus, during a given cycle up to 3 different instructions may be active at the same time, although each will be at a different stage of completion.

$$\text{The Speedup} = \frac{\text{Average instruction time (non-pipeline)}}{\text{Average instruction time (pipeline)}}$$

Pipelining has a major impact on the system memory because it leads to an increased no. of memory accesses.

The use of Harvard architecture where data and instructions lie in separate memory spaces promotes pipelining.

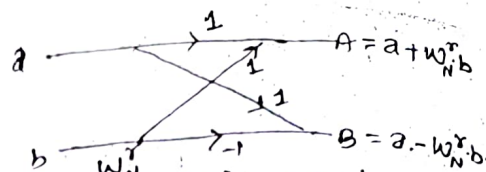
Bit-serial arithmetic;

Imp

inputs = 2 & b

$A = a$

outputs = ~~a & b~~
= A & B



Computational Complexity: Let us calculate the computational complexity required for one butterfly.

- i) To calculate any output (A or B), we need to multiply input 'b' by twiddle factor $-w_N^r$. Therefore, one complex multiplication is required for one butterfly.
- ii) To ~~calculate~~ ^{collect} o/p 'A', one complex addition is required, ~~for one butterfly~~ while to calculate 'B' one complex subtraction is required. But the computational complexity of both addition and subtraction is same. So we can say that for one butterfly two complex ~~additions~~ ^{operations} are required.
- iii) For - 8-point DFT, 4 butterflies are required in each stage. So for N-point DFT at each stage $\frac{N}{2}$ butterflies are required.
- iv) Also, three stages are required to compute - 8-point DFT.

In general, for N-point DFT, $\log_2(N)$ ~~stages~~ stages are required.

$$\text{Total complex multiplication} = \frac{N}{2} \log_2 N$$

$$\text{Total complex addition} = 2 \times \frac{N}{2} \log_2 N = N \log_2 N$$

$\therefore$ Speed improvement factor = $\frac{\text{Complex multiplication of direct computation}}{\text{Complex multiplication in FFT algorithm}}$

$$= \frac{N^2}{\frac{N}{2} \log_2 N} = \frac{2N}{\log_2 N}$$

No. of points.	Complex mult ⁿ in direct comp ⁿ	Complex mult ⁿ in FFT	Speed improvement factor
4	16	4	4
8	64	12	5.3
16	256	32	8
32	1024	80	12.8
64	4096	192	21.3
128	16384	448	36.6
256	65536	1024	64
		2206	113.8

Imp

Limit-Cycle Oscillations:

In the realization of digital filters, ~~the~~ the quantization inherent in the finite-precision arithmetic operations renders the system nonlinear. In recursive systems, the non-linearities due to the finite precision arithmetic operations often cause periodic oscillations to occur in the output, even when the input sequence is zero or some non-zero constant value. Such oscillations in recursive systems are called limit cycles and are directly attributable to round-off errors in multiplication and overflow errors in addition.

To illustrate the characteristics of a limit cycle oscillation let us consider a single pole system described by the linear difference equation.

$$y(n) = a y(n-1) + x(n)$$

where the pole is at $z=a$, the ideal system is realized ~~as~~ shown in fig. (a).

On the other hand, the actual system, which is described by the non-linear difference equation

$$v(n) = Q[a v(n-1)] + x(n)$$

is realized as shown in fig. (b).

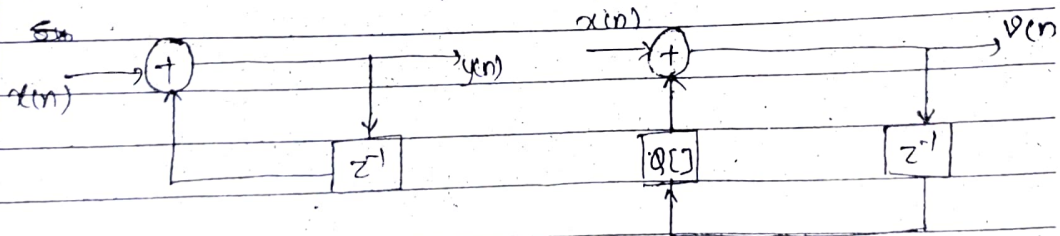


Fig. (a) ideal single-pole recursive system.

Fig. (b) Actual non-linear system

Suppose that the actual system in fig. (b) is implemented

with fixed point arithmetic based on 4 bits for the magnitude plus a sign bit. The quantization that takes place after multiplication is assumed to round the resulting product upward.

Let us list response of the actual system for four different locations of the pole $z=a$, and an input $x(n) = \beta \delta(n)$, where $\beta = 15/16$, which has the binary representation 0.111. Ideally, the response of the system should decay toward zero exponentially i.e.

$$y(n) = a^n \rightarrow 0 \text{ as } n \rightarrow \infty.$$

In the actual system, however, the response $y(n)$ reaches a steady-state periodic output sequence with a period that depends on the value of the pole. When the pole is positive, the oscillations occur with a period $N_p = 1$, so that the output reaches a constant value of $1/6$ for $a = 1/2$ and $1/8$ for $a = 3/4$. On the other hand, when the pole is -ve, the output sequence oscillates between +ve & -ve values ($\pm 1/6$ for $a = -1/2$ and $\pm 1/8$ for $a = -3/4$). Hence the period is $N_p = 2$.

These limit cycle occur as a result of quantization effects in multiplications. When the input sequence $x(n)$ to the filter becomes zero, the output of ~~the~~ filter then, after a no. of iterations, enters into the limit cycle. The output remains the limit cycle until another input of sufficient size is applied that drives the system out of the limit cycle. Similarly, zero input limit cycles occur from non zero initial conditions with the $x(n) = 0$.

→ Another possible limit cycle mode with zero input, which occurs as a result of rounding the multiplication, corresponding to an equivalent 2nd order system with poles at